Práctica 3 - Monitores

1. Se dispone de un puente por el cual puede pasar un solo auto a la vez. Un auto pide permiso para pasar por el puente, cruza por el mismo y luego sigue su camino.

```
Monitor  Puente
   cond cola;
   int cant= 0;

   Procedure entrarPuente ()
       while ( cant > 0) wait (cola);
       cant = cant + 1;
   end;

   Procedure salirPuente ()
      cant = cant – 1;
      signal(cola);
   end;
End Monitor;

Process Auto [a:1..M]
  Puente. entrarPuente (a);
  "el auto cruza el puente"
  Puente. salirPuente(a);
End Process;
```

a. ¿El código funciona correctamente? Justifique su respuesta.

No está bien ya que en Punte en la sección del while del procedimiento entrarPuente() la verificación de cant > 0 debería hacerse con un if para evitar el busywaiting.

b. ¿Se podría simplificar el programa? ¿Sin monitor? ¿Menos procedimientos? ¿Sin variable condition? En caso afirmativo, rescriba el código.

Se podría simplificar el programa utilizando un semáforo.

```
sem puente = 1;
Process Auto [a:1..M] {
        P (puente);
        // Cruzo
        V (puente);
}
```

c. ¿La solución original respeta el orden de llegada de los vehículos? Si rescribió el código en el punto b), ¿esa solución respeta el orden de llegada?

Tanto la solución original como la solución propuesta no respetan el orden de llegada, para que se cumpla el orden se debería utilizar una cola ordenada por id.

2. Existen N procesos que deben leer información de una base de datos, la cual es administrada por un motor que admite una cantidad limitada de consultas simultáneas.

a) Analice el problema y defina qué procesos, recursos y monitores/sincronizaciones serán necesarios/convenientes para resolverlo.

Proceso Lector -> acceder a la BD y salir de la BD

Monitor BD -> Permite el ingreso y egreso de los lectores a la BD.

b) Implemente el acceso a la base por parte de los procesos, sabiendo que el motor de base de datos puede atender a lo sumo 5 consultas de lectura simultáneas.

```
Process Lector [id: 0..N-1] {
        BD.Entrar ();
        // Uso la BD
        BD.Salir();
}

Monitor BD {
        int contador = 0;
        int espera = 0;
        cond cola;
```

```
        Procedure Entrar() {
                if (contador < 5) {
                        contador++;
                } else {
                        espera++;
                        wait (cola);
                }
        }
        Procedure Salir() {
                if (espera > 0) {
                        espera–;
                        signal(cola);
                } else {
                        contador–;
                }
        }
}
```

3. Existen N personas que deben fotocopiar un documento. La fotocopiadora sólo puede ser usada por una persona a la vez. Analice el problema y defina qué procesos, recursos y monitores serán necesarios/convenientes, además de las posibles sincronizaciones requeridas para resolver el problema. Luego, resuelva considerando las siguientes situaciones:

- Proceso Persona: las personas van a querer acceder a la fotocopiadora y luego deben salir.
- Monitor Fotocopiadora: solo puede ingresar una persona a la vez.

a) Implemente una solución suponiendo no importa el orden de uso. Existe una función Fotocopiar() que simula el uso de la fotocopiadora.

```
Process Persona [id: 0..N-1] {
        Fotocopiadora.imprimir();
}

Monitor Fotocopiadora {
```

```
        Imprimir();
}
```

b) Modifique la solución de (a) para el caso en que se deba respetar el orden de llegada.

```
Process Persona [id: 0..N-1] {
        Fotocopiadora.Pasar();
        Fotocopiar();
        Fotocopiadora.Salir();
}

Monitor Cajero{
        bool libre = true;
        cond cola;
        int esperando = 0;
        Procedure Pasar () {
                if(not libre) {
                        esperando ++;
                        wait(cola);
                }
                else libre = false;
        }
        Procedure Salir (){
                if(esperando > 0 ) {
                        esperando --;
                        signal(cola);
                }
                else libre = true;
        }
}
```

c) Modifique la solución de (b) para el caso en que se deba dar prioridad de acuerdo con la edad de cada persona (cuando la fotocopiadora está libre la debe usar la persona de mayor edad entre las que estén esperando para usarla).

```
Process Persona [id: 0..N-1] {
        int edad = ...;
        Fotocopiadora.Pasar(id, edad);
        Fotocopiar();
        Fotocopiadora.Salir();
}

Monitor Cajero{
        bool libre = true;
        cond espera[N];
        int idAux, esperando = 0;
        colaOrdenada fila;

        Procedure Pasar (intP, edad: in int) {
                if(not libre) {
                        insertar(fila, idP, edad);
                        esperando++;
                        wait (espera[idP]);
                }
                else libre = false;
        }
        Procedure Salir (){
                if(esperando > 0 ) {
                        esperando --;
                        sacar (fila, idAux);
                        signal(espera [idAux]);
                }
                else libre = true;
        }
}
```

d) Modifique la solución de (a) para el caso en que se deba respetar estrictamente el orden dado por el identificador del proceso (la persona X no puede usar la fotocopiadora hasta que no haya terminado de usarla la persona X-1).

```
Process Persona [id: 0..N-1] {
        Fotocopiadora.Pasar(id);
        Fotocopiar();
        Fotocopiadora.Salir();
}

Monitor Cajero{
        int proximo = 0;
        cond espera[N];

        Procedure Pasar (idP: in int) {
                if (idP != proximo) {
                        wait (espera[idP]);
                }
        }
        Procedure Salir (){
                proximo++;
                if (proximo < N) {
                        signal(espera[proximo]);
                }
        }
}
```

e) Modifique la solución de (b) para el caso en que además haya un Empleado que le indica a cada persona cuando debe usar la fotocopiadora.

```
Process Persona[id: 0..N-1]{
        fotocopiadora.llegue(id);
        //imprime
        fotocopiadora.terminar();
}


Process Empleado{
        while(true){
                fotocopiadora.sig();
```

```
        }
}

Monitor fotocopiadora(){
        Cola C;
        cond espera;
        cond personaEnEspera;
        cond imprimiendo;
        Procedure llegue(id: in int ){
                push(C, id);
                signal(personaEnEspera);
                wait(espera);
        }

        Procedure sig(id: in int){
                if(empty(C)){
                        wait(personaEnEspera);
                }
                pop(C,id);
                signal(espera);
                wait(imprimiendo);

        }

        Process terminar(){
                signal(imprimiendo);
        }
}
```

f) Modificar la solución (e) para el caso en que sean 10 fotocopiadoras. El empleado le indica a la persona cuál fotocopiadora usar y cuándo hacerlo

```
Process Persona[id: 0..N-1]{
        int idI;
        escritorio.llegue(id, idI);
        fotocopiadora[idI].imprimir();
```

```
        escritorio.terminar(idI);
}


Process Empleado{
        while(true) {
                escritorio.sig();
        }
}

Monitor escritorio(){
        Cola C;
        cond espera;
        cond personaEnEspera;
        cond esperaImpresoras;
        cond esperaAsignacion[N];
        Cola fotocopiadoras[10];
        vector personas[N];

        Process llegue(id: in int, idI: out int ){
                push(C, id);
                signal(personaEnEspera);
                wait(esperaAsignacion[id]);
        }

        Process sig(id: in int){
                if(empty(C)){
                        wait(personaEnEspera);
                }
                pop(C,id);
                if(fotocopiadoras.isEmpty()){
                        wait(esperaImpresoras);
                }
                int i = impresoras.pop();
                personas[id]=i;
                signal(esperaAsignacion[id]);
```

```
        }

        process terminar(id: in int){
                fotocopiadoras.push(id);
                signal(esperaImpresoras);
        }

}

Monitor fotocopiadora()[id:0..9]{
        Procedure imprimir(){
                // Imprime
        }
}
```

4. Existen N vehículos que deben pasar por un puente de acuerdo con el orden de llegada. Considere que el puente no soporta más de 50000kg y que cada vehículo cuenta con su propio peso (ningún vehículo supera el peso soportado por el puente).

V1:

```
Process Vehiculo [id: 0..N-1] {
        int peso = ...;
        Puente.Pasar(peso);
        // Cruza el puente
        Puente.Salir(peso);
}

Monitor Puente {
        bool libre = true;
        int esperando = 0;
        double pesoTotal = 0;
        cond esperaAutos;
        cond pesoEspera;
```

```
        Procedure Pasar (peso: in double) {
                if (not libre) {
                        esperando++;
                        wait (esperaAutos);
                }
                else libre = false;
                while (pesoTotal + peso > 500000) {
                        wait (pesoEspera);
                }
                pesoTotal += peso;
                if (esperando > 0) {
                        esperando–;
                        signal(esperaAutos);
                }
                else libre = true;
        }

        Procedure Salir (peso: in double) {
                pesoTotal -= peso;
                signal (pesoEspera);
        }
}
```

V2:

```
Process Vehiculo[id:0..N-1]{
        Puente.pasar(id,peso);
        //pasa
        Puente.termine(peso)
}

Monitor Puente {
        cond espera;
        Cola C[N];
        int pesoPuente=0;
```

```
        cond camiones;

        Procedure pasar(id: in int; peso: in int){
                if(not isEmpty(C)) or pesoPuente+peso>50000){
                        c.push(id, peso);
                        wait(espera);
                }
                else{
                        pesoPuente+=peso;
                }
        }

        Procedure termine(peso: in int){
                pesoPuente-=peso;
                if(not(isEmpty(C))){
                        id, peso=c.top(); // Podría ser .peek()
                        if(pesoPuente+peso <= 50000){
                                pesoPuente+=peso;
                                signal(espera);
                        }
                }
        }
}
```

5. En un corralón de materiales se deben atender a N clientes de acuerdo con el orden de llegada. Cuando un cliente es llamado para ser atendido, entrega una lista con los productos que comprará, y espera a que alguno de los empleados le entregue el comprobante de la compra realizada.

a) Resuelva considerando que el corralón tiene un único empleado.

V1:

```
Process Cliente[id:0..N-1]{
        text productos, comprobante;
        Gestion.llegue(productos, comprobante);
}

Process Empleado(){
```

```
        text productos;
        for int i: 1 .. N {
                Gestion.atender(productos);
                // Atiende al cliente
                text comp = generarComprobante(productos);
                Gestion.fin(comp);
        }
}

Monitor Gestion {
        int esperando = 0;
        cond esperandoCliente, despertarCliente, listadoListo,
esperandoComprobante, sali;
        text lista, comp;

        Procedure llegue(productos: in text, comprobante: OUT text){
                esperando++;
                signal (esperandoCliente);
                lista = productos;
                signal (listadoListo);
                wait (esperandoComprobante);
                comprobante = comp;
                esperando–;
                signal (sali);
        }

        Procedure atender(productos: OUT text){
                if (esperando == 0) {
                        wait (esperandoCliente);
                }
                signal (despertarCliente);
                wait (listadoListo);
                productos = lista;
        }

        Procedure fin(c: in text) {
```

```
                comp = c;
                signal (esperandoComprobante);
                wait (sali);
        }
}
```

V2:

```
Process Cliente[id:0..N-1]{
        text comprobante;
        text productos;
        Gestion.llegue(id, productos);
        Gestion.recibirComprobante(id, comprobante);
}

Process Empleado(){
        while(true){
                Gestion.sig(lista);
                text comprobante = generarC(lista);
                Gestion.darleComprobante(comprobante);
        }
}

Monitor Gestion{
        cond espera;
        Cola C[N];
        cond personaEsperando;
        cond colaEsperaComprobante;
        Cola esperaComprobante;
        text comprobantes[N];

        Procedure llegue(id: in int, productos: in text){
                push(C, id, productos);
                signal(personaEsperando);
        }

        Procedure sig(lista: out text){
```

```
            if c.isEmpty() wait (personaEsperando);
            else {
                productos=c.pop();
            }
        }

        Procedure recibirComprobante(id:in int , comprobante: out text) {
            push(C, id);
            wait(colaEsperaComprobante);
            comprobante=comprobantes[id];
        }

        Procedure darleComprobante(comprobante: in text){
            ¿NO TENDRÍA QUE VERIFICAR QUE LA COLA ESTÉ VACÍA?
            id=esperaComprobante.pop();
            comprobantes[id]=comprobante;
            signal(colaEsperaComprobante);
        }
}
```

b) Resuelva considerando que el corralón tiene E empleados (E > 1). Los empleados no deben terminar su ejecución.

c) Modifique la solución (b) considerando que los empleados deben terminar su ejecución cuando se hayan atendido todos los clientes.

6. Existe una comisión de 50 alumnos que deben realizar tareas de a pares, las cuales son corregidas por un JTP. Cuando los alumnos llegan, forman una fila. Una vez que están todos en fila, el JTP les asigna un número de grupo a cada uno. Para ello, suponga que existe una función AsignarNroGrupo() que retorna un número "aleatorio" del 1 al 25. Cuando un alumno ha recibido su número de grupo, comienza a realizar su tarea. Al terminarla, el alumno le avisa al JTP y espera por su nota. Cuando los dos alumnos del grupo completaron la tarea, el JTP les asigna un puntaje (el primer grupo en terminar tendrá como nota 25, el segundo 24, y así sucesivamente hasta el último que tendrá nota 1). Nota: el JTP no guarda el número de grupo que le asigna a cada alumno.

V1:

```
Process Alumno [id: 0..49]{
	int grupo, nota;
	Tare.llegar (id);
	Tarea.recibirGrupo (id, grupo);
	// Hacer tarea
	Tarea.recibirNota (grupo, nota);
}

Process JTP {
	int i, grupo;
	int vecContador [25] = ([25], 0);
	int puntaje = 25;
	Tarea.esperarAlumnos();
	for i: 1 .. 50 {
		Tarea.asignarTemas(AsignarNroGrupo());
	}
	for i: 1 .. 50 {
		Tarea.corregirTarea (grupo);
		// Corregir tarea
		vecContador [grupo]++;
		if vecContador[grupo] == 2 {
			Tarea.asignarPuntaje(grupo, puntaje);
			puntaje–;
		}
	}
}

Monitor Tarea {
	Cola espera, esperandoNota;
	int esperando = 0;
	int idAux;
	int vecGrupo [50], vecNota[25];
	cond espera [50], esperaNotas[50];
	cond deboCorregir;
```

```
Procedure llegar(id: in int) {
        esperando++;
        espera.push(id);
        if (esperando == 50) signal (esperaAsignar);
}

Procedure esperarAlumnos() {
        if (esperando < 50) wait (esperaAsignar);
}

Procedure recibirGrupo(id: in int, grupo: out int) {
        wait (espera[id]);
        grupo = vecGrupo[id];
}

Procedure asignarTemas (grupo: in int) {
        espera.pop(idAux);
        vecGrupo [idAux] = grupo;
        signal (espera[idAux]);
}

Procedure corregirTarea (grupo: out int) {
        if esperandoNotas.isEmpty() {
                wait (deboCorregir);
        }
        esperandoNotas.pop(grupo);
}

Procedure asignarPuntaje (grupo: in int, puntaje: in int) {
        vecNota[grupo] = puntaje;
        signal_all (esperandoNotas[grupo]);
}

Procedure recibirNota (grupo: in int, nota: out int) {
        esperandoNota.push(grupo);
        signal(deboCorregir);
```

```
            wait (esperaNotas[grupo]);
            nota = vecNota[grupo];
        }
}
```

V2:

```
Process Alumno [id: 0..49]{
        int grupo, nota;
        Tare.llegar (id);
        Tarea.recibirGrupo (id, grupo);
        // Hacer tarea
        Grupo[grupo].recibirNota (nota);
}

Process JTP {
        int i, grupo;
        int vecContador [25] = ([25], 0);
        int puntaje = 25;
        Tarea.esperarAlumnos();
        for i: 1 .. 50 {
            Tarea.asignarTemas(AsignarNroGrupo());
        }
        for i: 1 .. 50 {
            Tarea.corregirTarea (grupo);
            // Corregir tarea
            vecContador [grupo]++;
            if vecContador[grupo] == 2 {
                Tarea.asignarPuntaje(grupo, puntaje);
                puntaje–;
            }
        }
}

Monitor Tarea {
        Cola espera, esperandoNota;
```

```
        int esperando = 0;
        int idAux;
        int vecGrupo [50], vecNota[25];
        cond espera [50], esperaNotas[50];
        cond deboCorregir;

        Procedure llegar(id: in int) {
                esperando++;
                espera.push(id);
                if (esperando == 50) signal (esperaAsignar);
        }

        Procedure esperarAlumnos() {
                if (esperando < 50) wait (esperaAsignar);
        }

        Procedure recibirGrupo(id: in int, grupo: out int) {
                wait (espera[id]);
                grupo = vecGrupo[id];
        }

        Procedure asignarTemas (grupo: in int) {
                espera.pop(idAux);
                vecGrupo [idAux] = grupo;
                signal (espera[idAux]);
        }
}

Monitor Grupo [id: 0..24] {
        Cola esperandoNotas;
        int vecNota;
        cond esperaNotas[2];
        cond deboCorregir;

        Procedure corregirTarea (grupo: out int) {
                if esperandoNotas.isEmpty() {
```

```
                wait (deboCorregir);
            }
            esperandoNotas.pop(grupo);
        }

        Procedure asignarPuntaje (grupo: in int, puntaje: in int) {
            vecNota[grupo] = puntaje;
            signal_all (esperandoNotas[grupo]);
        }

        Procedure recibirNota (nota: out int) {
            esperandoNota.push(grupo);
            signal(deboCorregir);
            wait (esperaNotas[grupo]);
            nota = vecNota[grupo];
        }
}
```

7. Se debe simular una maratón con C corredores donde en la llegada hay UNA máquina expendedoras de agua con capacidad para 20 botellas. Además, existe un repositor encargado de reponer las botellas de la máquina. Cuando los C corredores han llegado al inicio comienza la carrera. Cuando un corredor termina la carrera se dirigen a la máquina expendedora, espera su turno (respetando el orden de llegada), saca una botella y se retira. Si encuentra la máquina sin botellas, le avisa al repositor para que cargue nuevamente la máquina con 20 botellas; espera a que se haga la recarga; saca una botella y se retira. Nota: mientras se reponen las botellas se debe permitir que otros corredores se encolen.

```
Process Corredor [id: 0..19] {
        Carrera.esperarInicio();
        // Corre
        Carrera.fin();
        Carrera.accederMaquina(); → PODRÍA SER DE MAQUINA?
        Maquina.usar();
        Carrera.dejarMaquina(); → PODRÍA SER DE MAQUINA?
```

```
}

Process Repositor {
	while (true) {
		Maquina.reponerBotellas();
	}
}

Monitor Carrera {
	bool libre = true;
	int cantC = 0, esperando = 0;
	cond cola, comienzoCarrera;

	Procedure esperarInicio() { // ESTÁ MAL RESOLVERLO TODO EN UN PROCESO?
		cantC++;
		if (cantidad == 20) signal_all (comienzoCarrera); // DEBERÍA
QUEDARSE DORMIDO TAMBIEN?
		else {
			wait (comienzoCarrera);
		}
	}

	Procedure accederMaquina() {
		if (not libre) {
			esperando++;
			wait (cola);
		} else {
			libre = false;
		}
	}

	Procedure dejarMaquina() {
		if (esperando > 0) {
			esperando–;
			signal (cola);
		} else {
```

```
                        libre = true;
                }
        }
}

Monitor Maquina () {
        int cantBotellas = 20;
        cond esperaBotella, repositor;

        Procedure usar() {
                if (cantBotellas == 20) {
                        signal (repositor);
                        wait (esperaBotella);
                }
                cantBotellas–; // Toma una botella
        }

        Procedure reponerBotellas() {
                if (cantBotellas > 0) wait (repositor);
                cantBotellas = 20;
                signal (esperaBotella);
        }
}
```

8. En un entrenamiento de fútbol hay 20 jugadores que forman 4 equipos (cada jugador conoce el equipo al cual pertenece llamando a la función DarEquipo()). Cuando un equipo está listo (han llegado los 5 jugadores que lo componen), debe enfrentarse a otro equipo que también esté listo (los dos primeros equipos en juntarse juegan en la cancha 1, y los otros dos equipos juegan en la cancha 2). Una vez que el equipo conoce la cancha en la que juega, sus jugadores se dirigen a ella. Cuando los 10 jugadores del partido llegaron a la cancha comienza el partido, juegan durante 50 minutos, y al terminar todos los jugadores del partido se retiran (no es necesario que se esperen para salir).

```
Process Jugador [id: 0..19] {
        int miEquipo = ...;
```

```
        int numeroC;

        Equipo[miEquipo].llegada(numeroC);
        Cancha[numeroC].llegada();
}

Monitor Equipo [id: 0..3] {
        int cant = 0, numCancha;
        cond espera;

        Procedure llegada (cancha: OUT int) {
                cant ++;
                if  (cant < 4) wait (espera)
                else  {
                        Admin.DarCancha(numCancha);
                        signal_all (espera);
                }
                cancha = numCancha;
        }
}

Monitor Admin {
        int cant = 0;
        Procedure DarCancha (num: OUT int) {
                cant ++;
                if  (cant <= 2) num = 0;
                else num = 1;
        }
}

Process Partido [id: 0..1] {
        Cancha[id].Iniciar();
        delay (50minutos); //se juega el partido
        Cancha[id].Terminar();
}
```

```
Monitor Cancha [id: 0..1] {
	int cant = 0;
	cond espera, inicio;

	Procedure llegada () {
		cant ++;
		if  (cant == 10) signal (inicio);
		wait (espera);
	}
	Procedure Iniciar () {
		if  (cant < 10) wait (inicio);
	}
	Procedure Terminar () {
		signal_all(espera);
	}
}
```

9. En un examen de la secundaria hay un preceptor y una profesora que deben tomar un examen escrito a 45 alumnos. El preceptor se encarga de darle el enunciado del examen a los alumnos cuando los 45 han llegado (es el mismo enunciado para todos). La profesora se encarga de ir corrigiendo los exámenes de acuerdo con el orden en que los alumnos van entregando. Cada alumno al llegar espera a que le den el enunciado, resuelve el examen, y al terminar lo deja para que la profesora lo corrija y le envíe la nota. Nota: maximizar la concurrencia; todos los procesos deben terminar su ejecución; suponga que la profesora tiene una función corregirExamen que recibe un examen y devuelve un entero con la nota.

```
Process Alumno [id: 0..44] {
	int nota;
	text examen;

	Aula.Llegada (examen);
	// Rendir examen
	Aula.Entrega (id, examen, nota);
}
```

```
Process Profesora {
	int i, idAlumno, nota;
	text examen;
	for i: 1 .. 45 {
		Aula.Examen (idAlumno, examen);
		nota = corregirExamen(examen);
		Aula.Corregido (idAlumno, nota);
	}
}

Process Preceptor {
	text enunciado = ...;
	Aula.DarEnunciado (enunciado);
}

Monitor Aula {
	int cant = 0, notas [45];
	text enunciado;
	cola C;
	cond eInicio, ePreceptor, eProfesora, eNota;

	Procedure Llegada (E: out text) {
		cant++;
		if (cant == 45) signal (ePreceptor);
		wait (eInicio);
		E = enunciado;
	}

	Procedure DarEnunciado (E: in text) {
		if (cant < 45) wait (ePreceptor);
		enunciado = E;
		signal_all (eInicio);
	}

	Procedure Entrega (id: in int, e: in text, N: out int) {
```

```
            C.push(id, e);
            signal (eProfesora);
            wait (eNota);
            N = notas [id];
        }

        Procedure Examen (idAlumno: out int, E: out text) {
            if c.isEmpty() wait (eProfesora);
            c.pop (idAlumno, E);
        }

        Procedure Corregido (id: in int, nota: in int) {
            notas[id] = nota;
            signal (eNota);
        }
}
```

10. En un parque hay un juego para ser usada por N personas de a una a la vez y de acuerdo al orden en que llegan para solicitar su uso. Además, hay un empleado encargado de desinfectar el juego durante 10 minutos antes de que una persona lo use. Cada persona al llegar espera hasta que el empleado le avisa que puede usar el juego, lo usa por un tiempo y luego lo devuelve.
Nota: suponga que la persona tiene una función Usar_juego que simula el uso del juego; y el empleado una función Desinfectar_Juego que simula su trabajo. Todos los procesos deben terminar su ejecución.

```
Process Persona [id: 0..N-1] {
        Admin.SolicitarUso();
        Usar_Juego();
        Admin.Liberar();
}

Process Empleado {
        while (true) {
            Desinfectar_Juego();
            Admin.Listo();
```

```
        }
}

Monitor Admin {
        cond ePersona, eEmpleado;
        int esperando = 0;
        bool libre = true;
        Procedure SolicitarUso() {
                if (not libre) {
                        esperando++;
                        wait (esperaAutos);
                }
                else libre = false;
        }

        Procedure Liberar() {
                signal (eEmpleado);
        }

        Procedure Listo () {
                if (esperando > 0) {
                        esperando–;
                        signal (ePersona);
                }
                else libre = true;
                wait (eEmpleado);
        }
}
```